

PRINCIPLES OF PROGRAMMING LANGUAGES / LESSON GUIDE

08 Runtime error

Modules 1-3: execution, exceptional control flow, and cleanup

1. Introduction & core concept

A runtime error is a failure encountered while a program is executing. Valid syntax and successful compilation cannot establish that every possible input or environmental condition will permit successful execution. Examples include unavailable resources, invalid conversions, failed checked accesses, and explicit violations of a function's preconditions.

Distinguish an exception from the broader category of runtime failure. A language may raise a catchable exception, return an error value, terminate a process, or leave behavior undefined for a violated rule. C++ undefined behavior is not a standard exception mechanism. Robust explanations identify the exact operation and guarantee rather than assuming every failure will be safely caught.

2. Under the hood: the mechanics

When an exception is raised or thrown, ordinary execution at that point stops. The implementation seeks a suitable enclosing handler, potentially leaving several active calls. Statements after the failing operation on that path are skipped. Cleanup constructs such as Python finally and JavaScript finally can run as control leaves a protected region [1-2].

C++ stack unwinding destroys fully constructed automatic objects in exited scopes when transferring to a matching handler [3]. This supports RAI: resource release is tied to object lifetime. Cleanup does not automatically roll back externally visible changes. Files written, messages sent, or shared values changed before failure may still require application-level recovery.

Learning outcomes

- Trace the failing operation and handler selection.
- Separate recovery from cleanup and rollback.
- Explain why catch-all handling does not make every program safe.

Examples progress from Python 3 to modern JavaScript to C++17. Code follows each language's native syntax. Bracketed citations refer to the final page.

3. CODE EXAMPLES / PYTHON

Simple: recover from a conversion failure

```
def double_text(text):  
    number = int(text)  
    return number * 2  
  
for text in ["4", "bad", "6"]:  
    try:  
        print(double_text(text))  
    except ValueError:  
        print("Invalid integer")  
    finally:  
        print("Attempt finished")
```

In-depth reasoning

Expected result: 8
Attempt finished
Invalid integer
Attempt finished
12
Attempt finished

For '4', int succeeds, the function returns 8, and print displays it. The finally suite then records completion. For 'bad', conversion raises ValueError before number receives a usable result, so the multiplication and function return are skipped. Control leaves double_text and reaches the matching handler in the caller.

After the handler prints Invalid integer, finally runs and the loop can process '6'. Placing the handler inside the loop isolates failure per item. A handler outside the entire loop would have a different continuation point. The example catches the expected conversion error instead of suppressing every possible exception.

Finally describes cleanup/control flow, not guaranteed completion under process termination, machine failure, or every conceivable event. A return or exception inside finally can also replace earlier control flow, so cleanup code should avoid hiding the original result or failure. The recorded message does not mean that conversion succeeded.

Compare the language models

JavaScript Number('bad') normally returns NaN rather than raising this Python exception. C++ conversion APIs may throw or return status depending on which API is used. Choosing a recovery strategy starts with the specific operation's contract.

Checkpoint: Why does the last input still run? The ValueError is handled within the loop iteration, and execution continues afterward.

3. CODE EXAMPLES / JAVASCRIPT

Intermediate: rejected asynchronous work

```
async function readCount(text) {
  const value = Number(text);
  if (!Number.isInteger(value)) {
    throw new TypeError("Expected an integer");
  }
  return value;
}

async function main() {
  try {
    const value = await readCount("bad");
    console.log(value);
  } catch (error) {
    console.log(error.name);
  } finally {
    console.log("Finished");
  }
}

main();
```

In-depth reasoning

Expected result: TypeError
Finished

`readCount` is `async`, so its result is represented by a promise. `Number('bad')` produces `NaN`; the explicit integer check rejects it by throwing `TypeError`. Within an `async` function, that thrown error becomes a rejection of the returned promise. The failure is created deliberately rather than by `Number` itself.

`Await` connects the promise's rejection to exceptional control flow inside `main`. The value assignment and following `log` are skipped, `catch` reports the name, and `finally` reports completion. Without `await`, a `try` around merely obtaining the promise would not catch a rejection delivered through that promise. Attaching an explicit rejection handler is another valid approach.

This function is a demonstration of propagation, not a complete input grammar. For example, `Number("")` is zero and passes the integer check. Separate validation requirements from the mechanism used to report a failure. The event loop schedules promise reactions; it does not create a separate operating-system thread for every `async` function.

Compare the language models

Python `async` code also requires attention to where an awaited failure is observed, while ordinary Python calls propagate synchronously. C++ exceptions propagate through the active call chain; futures, tasks, and threads require their own error-transport contracts.

Checkpoint: What is missing if `readCount` is called without awaiting or handling its promise? A defined place to observe and recover from its rejection.

3. CODE EXAMPLES / C++

Advanced: checked failure and RAI cleanup

```
#include <iostream>
#include <stdexcept>
#include <vector>

struct ScopeNotice {
    ~ScopeNotice() {
        std::cout << "Leaving scope\n";
    }
};

void inspect() {
    ScopeNotice notice;
    std::vector<int> values{10, 20};
    std::cout << values.at(5) << '\n';
}

int main() {
    try {
        inspect();
    } catch (const std::out_of_range&) {
        std::cout << "Index rejected\n";
    }
}
```

In-depth reasoning

Expected result: Leaving scope
Index rejected

`inspect` constructs `notice` and then `values`. The checked access `at(5)` is outside the two-element vector and throws `std::out_of_range`. No integer is inserted into the stream because argument evaluation fails first. Control seeks the handler in `main` and leaves `inspect`.

During this stack unwinding, fully constructed locals are destroyed in reverse construction order: `values` releases its storage, then `notice`'s destructor prints its message. The handler runs afterward and reports `Index rejected`. The demonstration assumes ordinary working output streams; real resource destructors should not let exceptions escape during unwinding.

Replacing `at(5)` with unchecked operator`[]` would not provide the same guarantee. Out-of-bounds vector indexing violates its access requirements and must not be presented as reliably throwing `out_of_range`. RAI helps release resources during defined exception propagation; it cannot repair arbitrary undefined behavior or guarantee recovery after forced process termination.

Compare the language models

Python list indexing raises `IndexError` for an invalid index; JavaScript reading an ordinary absent array element yields `undefined`. C++ deliberately offers checked and unchecked interfaces with different obligations. Similar indexing syntax does not establish equivalent failure behavior.

Checkpoint: Does unwinding undo modifications made before the throw? No. It destroys exited local objects; transactional rollback needs an explicit design.

4. CONSOLIDATION / REFERENCES

Reason, compare, and apply

The central lesson

Treat exceptions as structured information, preserve the operation and context that failed, and catch at a boundary able to recover. Paradigm Diagnostics isolates snippet execution in child processes, so a failed snippet can be reported without intentionally crashing the GUI. A nonzero exit status alone does not identify the language rule that caused failure.

Review questions with answer guidance

1. What should a handler avoid?

Silently converting all failures into success, losing diagnostics, or continuing with invalid state.

2. How do cleanup and recovery differ?

Cleanup releases resources; recovery establishes a usable next state. One does not imply the other.

A short learning activity

Predict each example's outcome before running it. Change one input or one operation, state the rule you expect to apply, and compare the observation with the prediction. Explain the earliest point where behavior changes. A matching output supports that particular execution; it does not prove general program equivalence.

References and further reading

[1] Python: exceptions and try statements. [Read source](#)

[2] ECMAScript: async function definitions. [Read source](#)

[3] C++ working draft: stack unwinding. [Read source](#)

References are official language/runtime documentation or the public C++ working draft. Links checked 30 September 2026. Examples and reasoning are original teaching material; implementation details and scheduling outcomes are identified where relevant.